Cost is a cocycle

No Lean in this one — it is design theory ahead of the mechanization, and it wears the honest badges to prove it. Its claims are exactly the ones a future track should turn green.

The machine's original cost model had a property so simple it was easy to mistake for essential: the cost of a run depended only on the **word** of instructions, not on the configuration it ran from. Cost was a homomorphism from the free monoid — cost of a concatenation is the sum, end of story. Two amendments now push against this: free `up` kept the property, but the proposed **dirty** pricing (a write-back is paid only when leaving a subtree that was written) does not: the same word can cost differently from different configurations. Is that the end of "algebraic"?

It is not, and this exposition says precisely what replaces it.

1 The cocycle law is functoriality

The machine's semantics is a (partial) action of the word monoid M on the set of configurations. A monoid is a one-object category BM, and the action is a functor $BM \to \mathrm{Set}$: the single object goes to the configuration set, each word to its (partial) transition function. The **action category** $\mathcal{C}$ is the Grothendieck construction — the category of elements — of this functor. Unwinding the definition: objects are configurations, and a morphism out of c is **not** a bare word but a pair (w, c) — a word runnable at c, welded to the configuration it runs from — with target $c \cdot w$ and composition

$$(w_2, c \cdot w_1) \circ (w_1, c) = (w_1 w_2, c).$$

The welding is the entire content of the construction: "the same word from a different configuration" becomes a **different morphism**, which is exactly the room a state-dependent cost needs to live in.

Now let $B\mathbb{N}$ be the one-object category with morphisms $(\mathbb{N}, +, 0)$. A cost model is a functor

$$\mathrm{cost} : \mathcal{C} \to B\mathbb{N}.$$

Because a morphism of $\mathcal{C}$ **is** a pair, a cost model is forced to be a function of pairs; write $\mathrm{cost}_c(w)$ for its value on the morphism (w, c) — subscript the source, nothing ad hoc. Functoriality over the composition law above unpacks to exactly two laws:

$$\mathrm{cost}(\mathrm{id}_c) = 0, \quad \mathrm{cost}_c(w_1 w_2) = \mathrm{cost}_c(w_1) + \mathrm{cost}_{c \cdot w_1}(w_2).$$

The shifted subscript is not an extra axiom — it only records that the second arrow of the composite starts where the first one landed. The second law is the **cocycle law** — the name comes from group cohomology (for a group action the category of elements **is** the translation groupoid, and functions with this law are its 1-cocycles) and from ergodic theory, where "cocycles over a dynamical system" are classical — including, pleasingly, cocycles over Vershik's adic transformations: the odometer keeps finding us.

The point of the definition: **state-dependent cost composes perfectly.** Nothing about the categorical semantics gets harder — a costed run is a morphism in the Writer-style graded setting (pairs of state-transform and price, composition adds prices), which is precisely the cost-graded monoid action the founding conversation already planned. The finite-presentation story also survives verbatim: define the cost table on finitely many cases — generator × **local observation** (the same locality read-branching already uses; dirtiness is one honest bit per node) — and functoriality extends it uniquely. The real cleanliness boundary was never "cost ignores state"; it is "cost reads only the local observation, from a finite table."

Static prices are a factorization. The Grothendieck construction comes with a projection $\pi : \mathcal{C} \to BM$, $(w, c) \mapsto w$ — forget the state, keep the word. A cost model is word-only precisely when it factors as $\mathcal{C} \to BM \to B\mathbb{N}$ through π: functors on BM are monoid homomorphisms into $(\mathbb{N}, +)$, the static prices; functors on $\mathcal{C}$ are the state-dependent ones. "Cost ignores state" stops being a property you check equation by equation and becomes a shape — does the functor factor? — and the conjecture below will say that write-back cost fails to factor even after correction by a coboundary.

2 Coboundaries are potential functions

Inside the cocycles live two distinguished classes. A cocycle is a **homomorphism** when it factors through π — cost blind to state; the base machine and the mount-distance surcharge both live here. And a cocycle is a **coboundary** when it is $(\delta\Phi)_c(w) = \Phi(c \cdot w) - \Phi(c)$ for a potential Φ on configurations. Coboundaries are cohomology's "trivial" cocycles — and they are exactly the potential functions of amortized analysis. Two cost models differing by a coboundary agree on every closed walk and differ only by boundary terms:

the cohomology class is the amortized cost.

Amortization-as-normalization (convo [21]) becomes: choose the cleanest representative of your class.

> `DESK-PROVED — not mechanized` **Free up is a change of representative**
>
> Let Φ = current depth of the head. On movement words,
>
> $$2 \cdot \mathrm{cost}_{\mathrm{free}} = \mathrm{cost}_{\mathrm{sym}} + \delta\Phi$$
>
> exactly (down: $2 = 1 + 1$; up: $0 = 1 - 1$). The free-`up` amendment did not change the cohomology class of movement cost — it chose a cleaner representative, and the boundary term $\Phi \le n$ is the familiar "moves ≤ 2·cost + initial depth" safety argument. Elementary; a mechanization would be a short induction on words.

3 Bandwidth is (conjecturally) a nontrivial class

Now the dirty model: `down` fills, a clean `up` discards for free, a dirty `up` pays its write-back. This cocycle reads one bit of local state, so it is **not** a homomorphism. The sharp question is whether it is secretly trivial — cohomologous to some word-only cost, its state-dependence absorbable into a potential.

> `UNFORMALIZED` **The write-back cocycle is not cohomologous to any homomorphism**
>
> No potential Φ and word-homomorphism h satisfy $\mathrm{cost}_{\mathrm{dirty}} = h + \delta\Phi$. Witness shape: m isolated single-bit writes cost $\Theta(m \cdot n)$ in write-backs, while the same multiset of operations arranged as one dense 2^k-block write costs $\Theta(2^k)$ — dirtiness **coalesces**. Equal word content, unboundedly different cost, comparable endpoints: no boundary term can absorb the gap.

If this holds — and the witness makes it hard to doubt — it is a theorem worth framing: **bandwidth cannot be statically priced.** Latency-like costs (addressing, movement, mount distance) are homomorphisms; write-back cost is genuinely history-dependent, and the cohomological language says so exactly. That real machines struggle to price bandwidth statically (rooflines, write amplification, "it depends on your access pattern") stops being folklore and becomes the statement that a certain cohomology class is nonzero.

> `DESK-PROVED — not mechanized` **Sparse–dense write asymmetry**
>
> Under dirty pricing: m isolated writes at pairwise tree-distant leaves cost $\Theta(m \cdot n)$ total (each pays its path home), while writing a full 2^k-subtree costs $\Theta(2^k)$ — each edge of the written region is dirty-crossed once, amortized $O(1)$ per written leaf. The write-side twin of the boundary spike; desk-level pending the dirty-bit mechanization.

4 What this buys the programme

Three earlier decisions become one picture. The machine's semantics was already a (partial) monoid action; its cost was already meant to be a grading (convo [7], [13]); amortization was already normalization (convo [21]). The cocycle view identifies all three: **a cost model is a functor to $B\mathbb{N}$; static prices are the homomorphisms; potentials are the coboundaries; amortized equivalence is cohomology; and the interesting resources are the nontrivial classes.** The mechanization needs none of this vocabulary — it proves cost equations word by word, as it already does — but the paper gets to say why those equations have the shape they have.